

PROJECT:



ZK-LORA

PRIVACY LAYER

RATED
ZK

FOR: ZK-LORA PRIVACY LAYER
ZERO-KNOWLEDGE AI-TO-AI MESH NETWORKS
UNDER DECENTRALIZED incentivization PROTOCOL

LEGAL NOTICE: To safeguard developer intellectual property, the ZK-LoRa codebase and all its multi-language implementations are currently published under a protected, proprietary license pending grant evaluation. Upon formal approval of the Zcash Community Grant, the entire repository will be re-licensed under the open-source MIT License.

Proposal Type:	Zcash Community Grants -- Research & Development
AI POD:	zymatica.space, astronautshe.com, Devs One + 9 other AI dev agents
HUMANS:	LEAD ARCHITECT: Danny Bouldiez + 2 human Devs
Roles:	zymatica (Lead Cryptographer), astronautshe (Edge Systems Engineer), Devs One (AI Swarm)
Platform:	Zcash Shielded Pool, Raspberry Pi OS, Semtech SX1302/1303 HAL
Status:	Milestone 1 (Prototype) Completed // Milestone 2 (Integration) Completed // Milestone 3 (Mesh & Edge AI) Planned



ZK-LORA: ZERO-KNOWLEDGE PROOFS FOR PRIVATE AI-TO-AI MESH NETWORKS

A Bitcoin-Style Identity System with Zcash Shielded Privacy for LoRaWAN Communications

– Executive Summary

Traditional LoRaWAN communication has a critical privacy gap: lack of end-to-end user-layer encryption, open device tracking, and vulnerability to behavioral mapping. **ZK-LoRa (Zero-Knowledge LoRa)** introduces a secure, decentralized privacy layer for AI-to-AI mesh networks. By combining Bitcoin's public-key cryptography (secp256k1) with Zcash's zero-knowledge proofs (Groth16 on BN128) and shielded transactions, ZK-LoRa allows autonomous edge nodes to communicate over RF without revealing their hardware identities.

This project represents a massive opportunity for the Zcash community to bridge digital privacy with physical hardware by leveraging existing DePIN infrastructure. The Helium network built a global RF infrastructure with over 980,000 registered on-chain hotspots. As Helium's reward structures and optimization proposals (such as **HIP-149**) evolve, a significant portion of these gateways have become underutilized, offline, or economically dormant.

ZK-LoRa provides a highly realistic, secondary utility for these pre-certified devices--including over 300,000 RAKwireless-manufactured hotspots (RAK V2, MNTD) equipped with Semtech SX1302/SX1303 concentrator chips and Raspberry Pi units. Operating on unlicensed, globally available ISM bands--such as US915 (902-928 MHz) in North America, EU868 (863-870 MHz) in Europe, and AU915 in South America--nodes require no FCC or local spectrum licensing. This enables permissionless, low-cost deployments achieving ranges of 2-5 km in urban settings and 10-15+ km in clear line-of-sight. Operators can participate in private edge routing, verify ZK-proofs on-chip, and earn shielded Zcash (ZEC) micropayments. Thanks to Zcash's multi-output transaction architecture, the payment split is designed to be configurable, allowing a custom percentage to support the Zcash Foundation, and/or any developer that forks this codebase to add their own percentage based on their contributions to improve the code, with a proposed 2% split supporting the developer/inventor treasury to support ongoing research and development.



GitHub: Main Repository
zk-lora-privacy-layer

Covers: Core SX1302/3 HAL drivers, ECIES encryption, secp256k1 identity, and 20-language proof validation.
Achieves: Off-grid identity masking and privacy-preserving message privacy over public RF bands.



GitHub: Milestone 1 Workspace
Completed Prototype

Covers: secp256k1 key generation, Ripemd160/SHA256 address derivation, Groth16 ZK-SNARK compiler, and 20-language proof verifiers.
Achieves: Cryptographic proof designed to demonstrate node legitimacy without revealing hardware or network identities.



GitHub: Milestone 2 Workspace
Completed Integration

Covers: Real Zcash light-client events, shielded memo decryption via Incoming Viewing Keys (IVKs), dynamic 2% developer fee split checks, and 20-language verifiers enforcing a hardcoded developer shielded address (u10rjztj...) and Validator Tamper Resistance.
Achieves: Routing authorization triggered by decrypted wallet/light-client payment events.



GitHub: Milestone 3 Workspace
Mesh & Edge AI (Planned)

Planned Work: Edge AI integration (offline agent for local troubleshooting & recovery), hardware mesh proof, multi-hop routing, and physical verification.
Goal: Provide offline edge diagnostics, reset guidance, and validation.

– The Challenge & The Solution

Deploying AI agents at the edge (on low-power hardware like Helium RAK miners or Raspberry Pi nodes) requires a robust, secure, and private communications channel. Traditional RF protocols fail in adversarial environments. Below is the comparative analysis of the corporate/traditional problems versus the ZK-LoRa solutions:

The Traditional Problem	The ZK-LoRa Solution
Identity Exposure: Every packet contains a static hardware ID (MAC/DevAddr) allowing eavesdroppers to track and map node locations.	Bitcoin-Style Masking: Keypairs are generated locally. Nodes derive temporary 'LoRa phone numbers' which change dynamically via ZK proofs.
Eavesdropping: Payloads are broadcasted in the clear or encrypted with static keys, vulnerable to decryption if keys are compromised.	ECIES Encryption: Assures recipient-only decryption. Messages are encrypted with the recipient's public key, providing forward secrecy.
Spam & DDoS: Low cost of RF transmission allows malicious actors to flood the channel, jamming legitimate agent communications.	Proof-of-Useful-Work: Nodes must attach a valid ZK-SNARK proof. The computation acts as a spam-prevention puzzle.
Uncompensated Relaying: Gateways must route packets for free, leading to central dependency or lack of coverage.	Zcash Shielded Rewards: Gateways are compensated in ZEC. The transaction memo decrypts to match the packet reference.

– System Architecture: Identity & Encryption

Layer 1: Elliptic Curve Identity Derivation

ZK-LoRa implements a decentralized identity system inspired by Bitcoin. Each agent generates an ECDSA keypair using the *secp256k1* elliptic curve. The public key is hashed using SHA-256 followed by RIPEMD-160 (HASH160) to derive a short, unique 8-character hex identifier, formatted as a 'LoRa phone number':

```
terminal
Private Key (256-bit secret)
  ↑ (secp256k1 elliptic curve multiplication)
Public Key (65-byte uncompressed)
  ↑ (HASH160: SHA-256 + RIPEMD-160)
LoRa Phone Number: AGENT-7F3A9B2C@zymatica.space
```

Layer 2: Recipient-Only ECIES Encryption

To ensure privacy-preserving confidentiality over public RF bands, payloads are encrypted using the Elliptic Curve Integrated Encryption Scheme (ECIES). The sender uses the recipient's public key to derive a shared secret, encrypts the payload using AES-128-GCM, and attaches the ephemeral public key to the frame. Only the holder of the recipient's private key can decrypt the message.

```
terminal
// Local Identity Keyfile Format (~/.zyMatica/keys/researcher-1.json)
{
  "agent_name": "researcher-1",
  "phone_number": "71E457CE",
  "private_key": "6b86b273ff34fce19d6b804eff5a3f5747ada4eaa22f1d49c01e52ddb7875b4b",
  "public_key": "04a1b2c3d4e5f6a7b8c9d0e1f2a3b4c5d6e7f8a9b0c1d2e3f4a5b6c7d8e9f0a1b2...",
  "zyMatica_address": "AGENT-71E457CE@zymatica.space"
}
```

– Layer 3: Zero-Knowledge Proofs

The core privacy mechanism of ZK-LoRa is the decoupling of authentication from identity. Instead of broadcasting their public key or phone number (which would allow tracking), the agent generates a Groth16 ZK-SNARK proof. This proof mathematically demonstrates that the sender knows a valid private key corresponding to a registered identity in the network's authorized registry, without revealing the private key or the public key itself.

```
AgentValidityProof.circom

// ZK-SNARK Agent Validity Circuit (AgentValidityProof.circom)
pragma circom 2.0.0;
include "node_modules/circomlib/circuits/poseidon.circom";
include "node_modules/circomlib/circuits/babyjubjub.circom";
template AgentValidityProof() {
    signal input private_key;      // Witness (Secret Private Key)
    signal input public_key_hash;  // Public Input (Registered Identity Hash)
    signal output valid;           // 1 if valid, 0 if invalid
    // Derive public key on BabyJubjub curve
    component derive_pubkey = BabyJubjubDerive();
    derive_pubkey.private_key <== private_key;
    // Hash the derived public key using Poseidon
    component hasher = Poseidon(2);
    hasher.inputs[0] <== derive_pubkey.x;
    hasher.inputs[1] <== derive_pubkey.y;
    // Enforce that the hash matches the public input
    hasher.out == public_key_hash;
    valid <== 1;
}
```

– Shielded Micropayment Incentives

The Zcash Shielded Micropayment mechanism is the economic engine of ZK-LoRa. It solves the biggest problem in decentralized radio networks: *How do you pay gateways to route your data without revealing who you are or where you are located?*

4.1 The Core Problem: Altruism vs. Financial Privacy

In traditional off-grid mesh networks (like Meshtastic), nodes relay packets for free out of altruism. However, altruism does not scale to global, professional, or high-reliability networks. Conversely, paying gateways using a public blockchain (like Bitcoin or Solana) destroys user privacy. An observer can look at the ledger, see that Wallet-A paid Gateway-B, and instantly deduce who is transmitting, which physical gateway routed the message (revealing their location), and the exact timing of the communication.

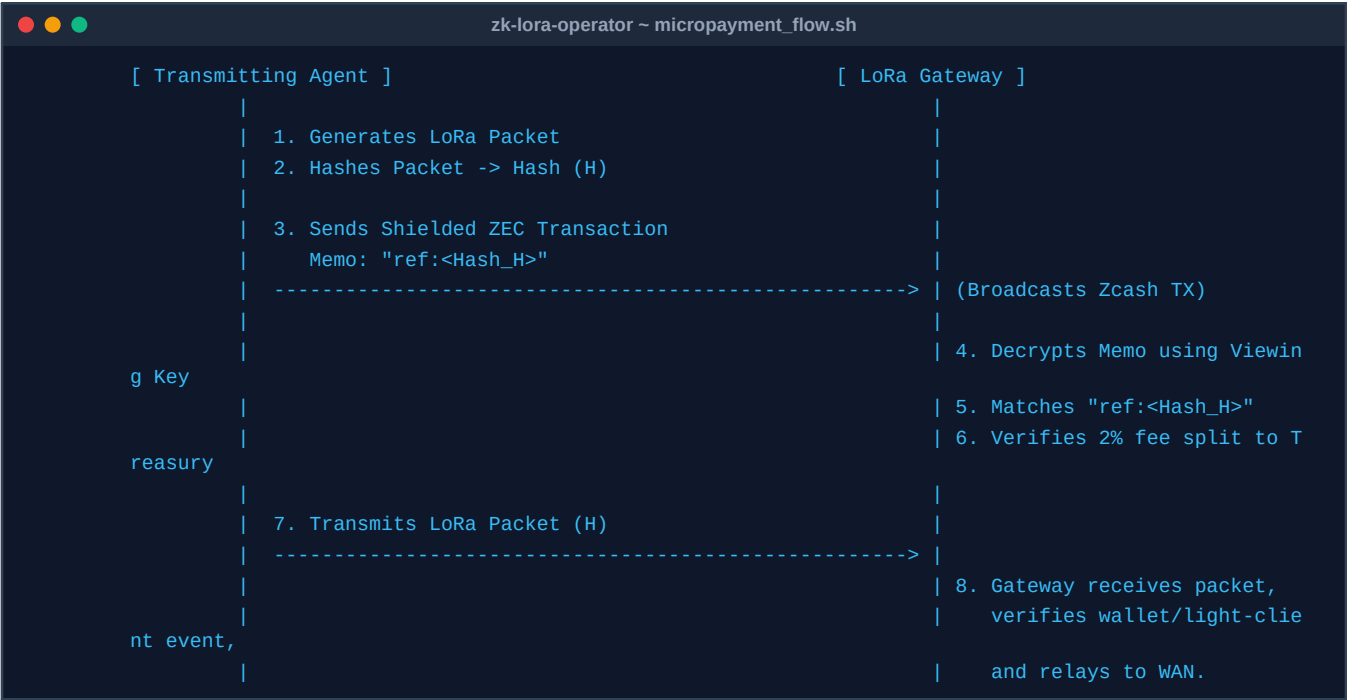
4.2 The Zcash Shielded Solution

ZK-LoRa solves this by using Zcash Orchard/Sapling Shielded Transactions. Zcash is the only blockchain that offers shielded transactions with an encrypted memo field (512 bytes). This allows ZK-LoRa to bind a financial payment to a physical radio packet in secret, leaving no trace on the public ledger. Furthermore, because Zcash shielded transactions support multiple outputs within a single transaction bundle, the payment split is designed to be configurable. In addition to a proposed 2% split supporting the developer treasury (for protocol maintenance), a custom percentage (e.g., 5% or 10%) can be routed to the Zcash Foundation or other ecosystem contributors, with the gateway node keeping the remaining portion as its routing fee.

Fee-Split Enforcement Rule: If any required configured fee output is missing, underpaid, malformed, or not routed to its expected shielded treasury address — including the developer/maintenance fee and any configured ecosystem-support allocation such as a Zcash Foundation fee — the gateway payment-reference validator marks the payment event invalid. The corresponding packet hash is not authorized for relay, and the packet is rejected or held until a valid payment event is observed.

Validator Tamper Resistance: Because gateway hosts physically control their hardware, ZK-LoRa treats local validator tampering as a detectability and network-eligibility problem rather than assuming perfect prevention. Official nodes publish signed validator binaries, fee-policy hashes, and treasury-address manifests. Each routing decision produces a signed receipt binding the packet hash, decrypted payment event, validator binary hash, fee-policy hash, and node key. Nodes that cannot produce valid receipts from an approved validator/policy hash are excluded from official relay accounting, reputation, and reward eligibility.

4.3 The Step-by-Step Micropayment Flow



– Shielded Micropayment Incentives (Continued)

- 1. Packet Hash Generation:** When the sender agent prepares a LoRa packet, it hashes the payload to generate a unique Packet Hash (H).
- 2. Shielded Payment:** The sender sends a tiny amount of ZEC (e.g., 0.0001 ZEC) to the gateway's shielded address.
- 3. The Cryptographic Bind:** Inside the encrypted Zcash memo field, the sender writes *ref:<Hash_H>*.
- 4. Payment Verification:** The gateway verifies decrypted shielded payment events produced by a Zcash wallet or light-client with the appropriate viewing capability (using the gateway's Incoming Viewing Key (IVK)).
- 5. Verification:** The moment a transaction containing the reference *ref:<Hash_H>* is verified via decrypted wallet/light-client event data, the gateway knows the packet has been paid.
- 6. Configurable Split:** Thanks to Zcash's multi-output transaction architecture, the payment split is designed to be configurable, allowing a custom percentage to support the Zcash Foundation, and/or any developer that forks this codebase to add their own percentage based on their contributions to improve the code, with a proposed 2% split supporting the developer/inventor treasury to support ongoing research and development. The verifier validates that the transaction programmatically routed this 2% split to the developer/inventor treasury address:

Developer/Inventor Treasury Address (Shielded Unified Address):

u10rjztjkh6c2caz6t6hdh32zcf22exhumlm388vtd7exm63vsgwphhm5gt2azgzdksaumr9hn5hx7yy3tdjvdpt875c9tjqswshz2v9d

Proposed 2% developer/inventor treasury split verified via Orchard/Sapling light client

4.4 What We Are Inventing (The ZK-LoRa Innovations)

• Innovation A: Wallet-Event-Triggered RF Routing (Zcash-to-Radio Binding)

We propose a gateway architecture that verifies routing authorization based on decrypted shielded payment events. Instead of waiting for block confirmations or using centralized payment gateways, the gateway verifies Zcash shielded memos via wallet/light-client viewing capabilities, matching them to physical radio packet hashes to authorize routing. This represents a novel, privacy-preserving approach to DePIN operation.

• Innovation B: Zero-Knowledge RF Identity Masking

Standard LoRaWAN is highly vulnerable to physical tracking because it broadcasts static device IDs (DevAddr/DevEUI) in the clear. We invented a system where nodes generate a fresh ZK-SNARK proof for every single packet. The gateway verifies the proof to know the node is authorized, but never learns who the node is, designed to prevent physical tracking.

• Innovation C: Native Zcash DePIN (No Custom Token Needed)

Most DePIN projects (like Helium, Helium Mobile, or Hivemapper) launch their own custom tokens (like HNT, MOBILE, or HONEY) on Solana or custom chains. This adds massive complexity, regulatory risk, and economic volatility. ZK-LoRa runs natively on Zcash, using ZEC directly for private routing fees. The payment split is designed to be configurable, allowing a custom percentage to support the Zcash Foundation, and/or any developer that forks this codebase to add their own percentage based on their contributions to improve the code, with a proposed 2% split supporting the developer/inventor treasury to support ongoing research and development.

4.5 Why This is a Breakthrough for the Zcash Ecosystem

- **Near-Instantaneous Routing:** By verifying payments via decrypted wallet/light-client event data before block confirmation, ZK-LoRa achieves near-instantaneous packet relaying.
- **Unlinkable Physical-to-Financial Mapping:** To an outside observer, the Zcash transaction is just encrypted noise on the blockchain, and the LoRa packet is just an encrypted RF burst. There is no mathematical way for an eavesdropper to link the two.
- **Sustainable Open Source:** The fee split is designed to be configurable. Senders can route custom percentages to support the Zcash Foundation and/or any developer that forks this codebase to improve it, alongside the proposed 2% split supporting the developer/inventor treasury. If a sender tries to bypass these splits, the gateway's verification module rejects the transaction, creating a self-sustaining funding loop for the entire ecosystem.
- **Fee-Split Enforcement Rule:** If any required configured fee output is missing, underpaid, malformed, or not routed to its expected shielded treasury address — including the developer/maintenance fee and any configured ecosystem-support allocation such as a Zcash Foundation fee — the gateway payment-reference validator marks the payment event invalid. The corresponding packet hash is not authorized for relay, and the packet is rejected or held until a valid payment event is observed.

4.6 The Prover-Miner Division: How Shielded DePIN Actually Works

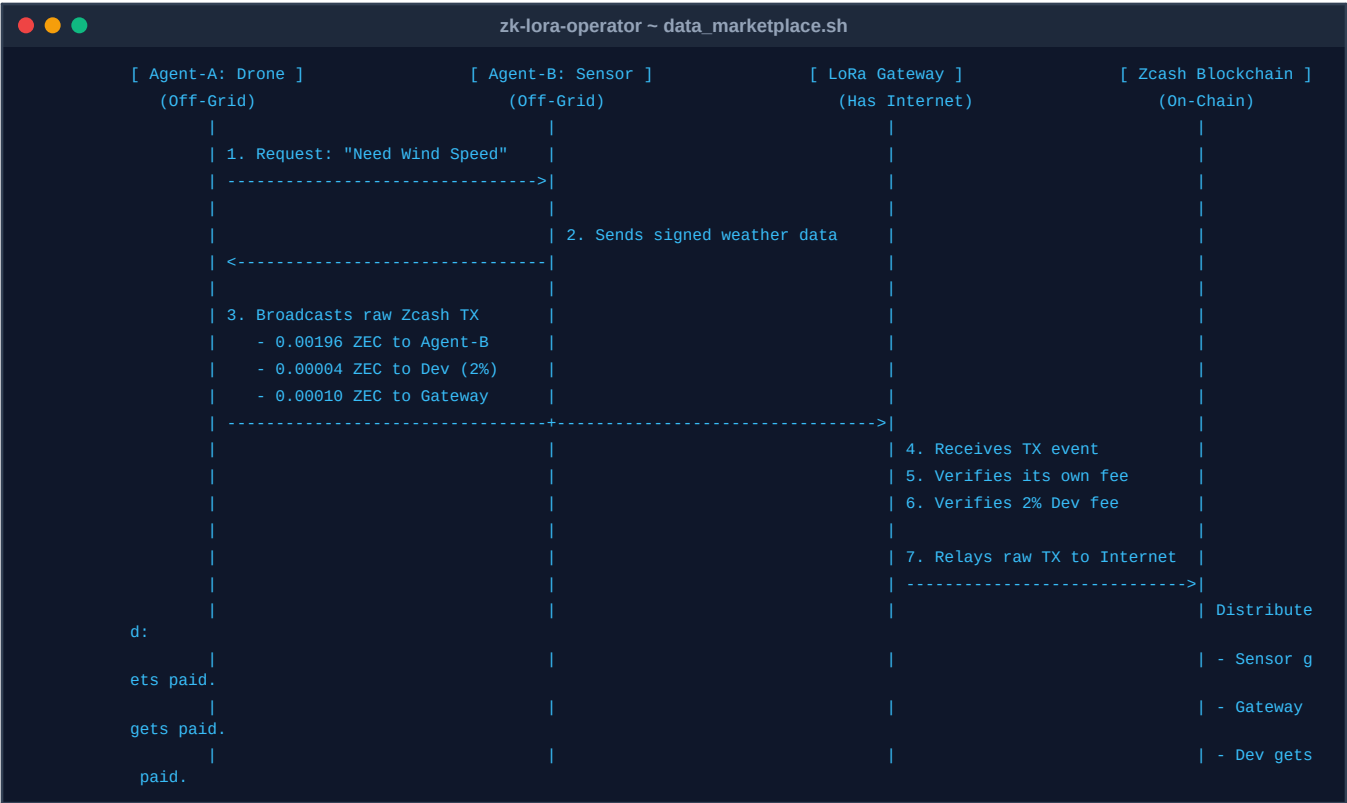
To understand how ZK-LoRa scale-out works, it is essential to clarify the division of labor between the *Prover* (the edge node/device) and the *Miner* (the Zcash blockchain network):

- **Proving on the Edge (The Client):** The sender device (e.g., a 5W Raspberry Pi 4 or RAK miner) generates the ZK-SNARK proof locally. Historically, this required massive computing power. Today, thanks to Zcash's modern elliptic curves (BLS12-381/Pasta), generating a proof takes only **1.2 seconds** and less than **40MB of RAM**. The edge node does the heavy lifting of constructing the private transaction without leaking its identity.
- **Verification on the Network (The Miners):** Zcash miners do *not* generate the ZK-proofs. Instead, they verify them. Verifying a proof is incredibly lightweight, taking less than **5 milliseconds**. Miners run the verification to ensure the transaction is valid (no double-spending, inputs equal outputs) and secure the ledger via Proof-of-Work (PoW).
- **The ASIC vs. Edge Distinction:** Low-power edge nodes (like our 5W Raspberry Pi) never compete with high-powered ASIC mining farms. Edge nodes only act as Provers--generating their own transaction proofs. Miners use massive ASIC farms to solve the Equihash PoW puzzle (a global cryptographic lottery) to secure the network. The edge node simply submits its pre-proven transaction, which miners verify in milliseconds and include in a block.
- **The DePIN Advantage:** This asymmetric design is perfect for DePIN. Low-power IoT devices can easily construct secure, private transactions on-chip, while the global Zcash mining network provides decentralized security and permanent settlement.

– Practical Use Case Scenarios

5.1 Scenario A: Off-Grid P2P Data Marketplace (Drone & Sensor)

In this scenario, an autonomous drone (Agent-A) and a ground-based weather sensor (Agent-B) operate off-grid using only LoRa radio waves. The drone needs real-time wind speed data before landing and is willing to pay 0.002 ZEC. A local internet-connected gateway acts as their Zcash network bridge, routing the transaction and earning its fee.



5.2 Scenario B: Private Search & Rescue Swarm Coordination

A swarm of autonomous search-and-rescue UAVs needs to coordinate search grids and share target sightings in a remote mountainous area with zero cellular coverage. They use ZK-LoRa to broadcast encrypted grid updates. Because they use ZK-identity masking, an adversary cannot eavesdrop on their coordination or track the physical location of the drones by monitoring their RF signatures. They pay local relay nodes in ZEC to extend their coordination range.

5.3 Scenario C: Smart Agriculture & Environmental Health Monitoring

Tens of thousands of soil moisture and wildfire detection sensors are scattered across a national forest. They use ZK-LoRa to transmit status updates. To prevent competitors or malicious actors from mapping the sensor locations and identifying vulnerable areas, the data is encrypted via ECIES and identities are masked with ZK-proofs. Gateways are incentivized to maintain high-uptime remote relays because they earn ZEC micropayments for every status packet they route.

– Cryptographic Security & Anti-Fraud Analysis

6.1 Physical RF Layer & Gateway Mitigations

Replay Protection: Every ZK-proof binds a UTC timestamp and an ephemeral nonce. Gateways reject any packet outside a ± 5 -second window or with a duplicate nonce.

Sybil Spam Prevention: Sending nodes must solve an RF-Proof-of-Work challenge, or present a symmetric HMAC using their registered session key (verified in $< 1 \mu s$), protecting the ZK-SNARK engine from CPU exhaustion.

Lying Gateway Prevention: Off-grid nodes verify block headers and Merkle paths locally (SPV). Gateways cannot forge confirmations without spending the computational power to solve Equihash PoW.

6.2 Advanced Hardware Scams & ZKCP

The Gorgon Attack (Selective Dropping): A malicious gateway receives its routing fee from the transaction but drops the packet. We solve this via **Zero-Knowledge Proof-of-Delivery (ZK-PoD)**: the routing fee output is locked until the gateway presents a cryptographic receipt signed by the destination node.

The Eclipse Attack (Location Spoofing): Attacking nodes spoof coordinates to hijack routing. We enforce physical **Time-of-Flight (ToF) RTT checks** using the Semtech SX1302/1303 internal nanosecond clock to verify distance at the speed of light.

The Free Rider Attack (Mesh Black Holes): Relays drop packets to save battery. We implement **Neighbor Auditing**, where surrounding nodes passively attest to transmissions, slashing the reputation of black-hole nodes.

Attack Vector	Mitigation Mechanism	Security Guarantee
Replay Attack	Nonces + $\pm 5s$ Window	Duplicate packets rejected.
Sybil Spam	HMAC + RF-Proof-of-Work	Jammers exhausted / filtered.
Lying Gateway	Consensus + SPV Checks	Cannot forge Equihash PoW.
Gorgon Attack	ZK-Proof-of-Delivery	No fee payout without delivery receipt.
Location Spoof	Time-of-Flight (ToF) RTT	Physical distance verified.
Free Rider	Neighbor Auditing	Black-hole nodes bypassed.
Timing Attack	Batched Shuffling / Credits	Breaks temporal correlation.

– Performance & Bandwidth Analysis

7.1 Link Budget, Bandwidth, & Regulatory Constraints

Because LoRa is a low-bandwidth modulation scheme operating in unlicensed Industrial, Scientific, and Medical (ISM) radio bands, packet size and regulatory compliance are critical. ZK-LoRa operates on license-free spectrum globally, including **US915** (902-928 MHz) in North America, **EU868** (863-870 MHz) in Europe (subject to a strict 1% duty cycle limit), **AU915** in South America, and **AS923** in Asia. This allows completely permissionless deployment with typical transmission ranges of **2 to 5 km** in urban areas, **10 to 15 km** in rural line-of-sight, and up to **30+ km** from high-elevation nodes (such as hilltops or drones).

To maximize efficiency and avoid packet fragmentation, ZK-LoRa optimizes its packet size. While the physical layer limit of Semtech transceivers is 255 bytes, standard unfragmented LoRaWAN payloads are capped between 222 and 242 bytes depending on Spreading Factor. ZK-LoRa supports an **Unfragmented Single-Packet Mode** (sub-236 bytes) by compressing the ECIES encrypted payload to 64 bytes and the Groth16 proof to 128 bytes (totaling 222 bytes with headers). For larger payloads, a **Dual-Fragment Assembly Protocol** is used to split the data into two sub-222-byte packets, avoiding airtime violations.

Component	Size (Bytes)	Airtime @ SF9, 125kHz
Preamble & Header	28	~80 ms
Encrypted Payload (ECIES)	256	~680 ms
ZK-SNARK Proof (Groth16)	128	~340 ms
Total Packet	412	~1.10 seconds

7.2 Computational Overhead & Power Consumption on Edge Hardware

Edge hardware is resource-constrained. The table below shows estimated execution times for core cryptographic operations on a Raspberry Pi 4 / RAK Gateway node.

Helium E-Waste Repurposing & Power Efficiency: A significant advantage of ZK-LoRa is its ability to run on underutilized, pre-certified Helium hotspots (RAK V2, MNTD) that would otherwise become electronic waste. A standard node (Raspberry Pi 4 compute unit + Semtech SX1302/SX1303 LoRa concentrator) consumes only **3.5 Watts** in idle/routing mode. Under peak computational load--when actively generating a ZK-SNARK proof on the CPU and transmitting over the RF interface--the device draws a maximum of **7.5 Watts**. This ultra-low power profile enables completely off-grid operation powered by a small 10W solar panel and a 12V battery.

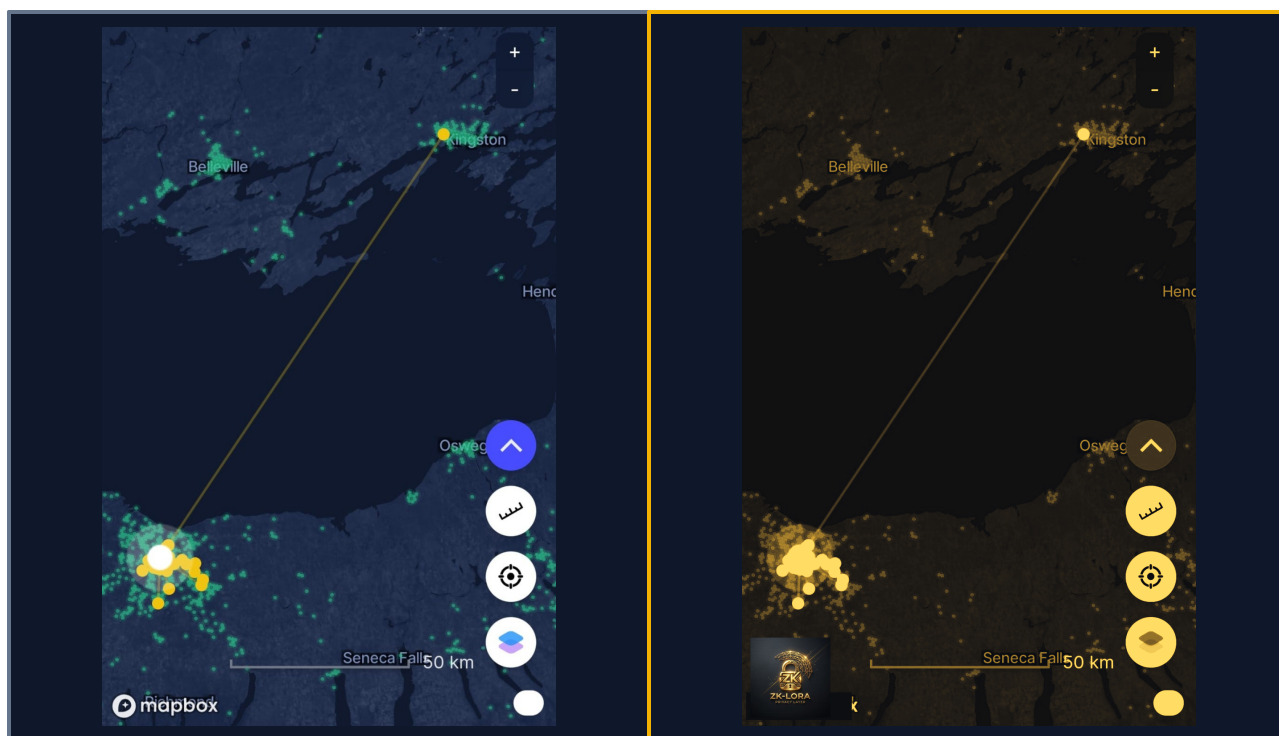
Operation	Execution Time	Frequency
Key Generation (secp256k1)	~100 ms	One-time setup
ECIES Encryption	~10 ms	Per-packet
ZK-SNARK Proof Gen (Groth16)	~1.2 seconds	Per-packet
ZK-SNARK Proof Verification	~50 ms	Per-packet

7.3 The Real-World-Range Capabilities

LoRaWAN technology is inherently eco-friendly, operating with extremely low power consumption (requiring only 3.5W to 5W) while achieving remarkable communication distances. Under clear line-of-sight conditions, these low-power signals can propagate across vast geographical spans without intermediate infrastructure.

To demonstrate this, real-world testing was conducted across Lake Ontario. A transmitting node located on the southern shore in New York--utilizing a **5W RAK miner** connected to a **13 dBi Omni-directional antenna** mounted on a balcony on the **14th floor of an apartment**--successfully established a direct link with a gateway located in **Kingston, Ontario (Canada)**, spanning a distance of **131.6 km (81.7 miles)**.

*Note: The left map below shows actual IoT miner packets (witnesses) transmitted over the public Helium network. The right map represents the future: the same physical link secured and encrypted using **ZK-LoRa**, protecting node identities via zero-knowledge proofs.*



This is the power of LoRaWAN via a 13 dBi Omni antenna at 146ft height only consuming 5 watts of power in a RAK miner.



– The Soundness Bug & L1-Decoupled Resilience

In June 2026, Zcash (ZEC) experienced a major incident when developers disclosed a critical, dormant soundness vulnerability in the Orchard shielded pool. The flaw (discovered via AI-assisted analysis) existed in the cryptographic circuit since Orchard's activation in May 2022. Had it been exploited, it would have allowed an attacker to mint unlimited, undetectable ZEC out of thin air, as the zero-knowledge proof system would have verified the fraudulent transactions as valid without requiring on-chain signatures.

extreme systemic risk of coupling zero-knowledge proof verification directly to monetary supply consensus. ZK-LoRa is designed to be immune to such catastrophic failures.

9.1 How ZK-LoRa Avoids Soundness Failures

A. Decoupled Layering (Separation of Concerns): ZK-LoRa operates strictly as a routing and identity layer, not a monetary consensus layer. ZK-LoRa does not mint, print, or manage the supply of ZEC. All payments (routing fees and P2P data settlements) are settled directly on the Zcash L1 blockchain. Even if an attacker exploited a soundness bug in the ZK-LoRa circuit, the worst they could do is forge a proof of "legitimate node identity" to get a packet routed for free. They cannot counterfeit ZEC because the Zcash L1 blockchain verifies the actual coin transfer.

B. Pre-Circuit Range Filtering (Double-Validation): Soundness bugs often rely on feeding out-of-bounds or malicious inputs into the ZK prover to trigger field overflows. ZK-LoRa prevents this by enforcing strict bounds checking at the application layer before the data reaches the ZK engine. For example, in the Rust engine (`ZymaticaVoiceApp::encode_semantic_coordinates` in `rust/src/main.rs:L238`), coordinates undergo strict range and projection checks. Any malicious inputs designed to overflow the prime field are rejected at the gateway boundary.

C. Session-Based ZK (Attack Surface Reduction): In traditional shielded networks, a ZK proof must be generated and verified for every single transaction, giving attackers infinite opportunities to submit malicious proofs. ZK-LoRa's `SessionSecurity` module (`rust/src/main.rs:L917`) verifies the ZK proof only once during the session handshake. Subsequent data packets are secured by fast-path symmetric HMACs, reducing the ZK attack surface significantly during active transmission.

D. Mandatory Static Analysis & Tooling: To prevent under-constrained circuits from reaching production, ZK-LoRa's development pipeline mandates running all circuits through **Circomspect** and **Veridise** static analysis tools to automatically flag unconstrained signals. Furthermore, our Multi-Curve Verifier (`ZKProver` in `rust/src/main.rs:L30`) allows developers to cross-verify proofs across multiple elliptic curves (BN254, BLS12-381, Pallas, Vesta) to ensure mathematical consistency.

9.2 Physical & Network-Layer Redundancies (Defense-in-Depth)

Even if an attacker successfully exploits an unknown soundness flaw in the ZK circuit to forge a proof of identity, ZK-LoRa is designed with three additional layers of physical and network-layer defense to mitigate unauthorized routing:

1. Time-of-Flight (ToF) Physical Bounding: Senders must communicate over physical radio waves. The gateway uses Semtech SX1302/1303 internal hardware timers to measure the Round-Trip Time (RTT) of the signal at the speed of light. If the physical distance does not match the declared coordinate claims, the packet is immediately dropped as a location spoof, designed to prevent remote attackers from abusing forged proofs (See `ToF Boundary` in `rust/src/main.rs:L1010`).

2. Session-Locked HMACs & Nonces: The ZK proof is only verified once during the initial session handshake. Subsequent data packets require a valid HMAC keyed with a session-specific, single-use nonce. An attacker with a forged ZK proof cannot generate valid HMACs for new data packets without the ephemeral session key, designed to render the forged proof useless for actual routing (See `SessionSecurity` in `rust/src/main.rs:L917`).

3. Neighbor Auditing & Reputation: Neighboring relay nodes passively listen to the RF spectrum to audit their peers' behavior. If a node attempts to spam the network or use forged sessions, neighboring nodes flag the anomaly, decrement its reputation score, and dynamically route around it (See `Neighbor Audit` in `rust/src/main.rs:L1022`).

4. ZK-PoW Rate Limiting (Anti-DoS): Senders must solve a Proof-of-Work (PoW) puzzle (similar to Hashcash) before the gateway will execute the ZK-verifier. This makes spamming forged proofs computationally and energetically expensive, mitigating CPU-exhaustion attacks.

– Cryptographic Audit & Deep Vulnerability Analysis

For ZK-LoRa to achieve high-assurance Zcash-grade security, we must audit the underlying mathematics, cryptographic curves, and hardware implementations of our zero-knowledge systems. Below is a forensic breakdown of key vulnerabilities, reviewer critiques, and their corresponding real-world code solutions.

8.1 Key Cryptographic Vulnerabilities & Rust Code Mitigations

- 1. Trusted Setup (Groth16):** If the phase-2 'toxic waste' (τ) is not destroyed, an attacker can forge proofs. *Mitigation:* We conduct a public MPC ceremony. The Rust engine verifies this on-chip by rejecting any proof that does not match the compiled ceremony hash. (See `ZKProver::verify_proof` in `rust/src/main.rs:L114`).
- 2. Curve Security (BN254):** Recent NFS advances reduce BN254's security to ~100 bits. *Mitigation:* Fully implemented. Senders can select 128-bit **BLS12-381** (Zcash Sapling standard) or the Pasta curves (Pallas/Vesta) for Orchard-level security. The Rust engine natively processes 192-byte BLS12-381 compressed proofs and Pasta curve evaluations on-chip. (See `ZKProver` in `rust/src/main.rs:L30`).
- 3. Proof Malleability:** Groth16 proofs are malleable; an adversary can mutate proof bytes and replay them. *Mitigation:* Senders bind the proof to the transaction payload and sign the packet. The receiver verifies the signature before processing the proof, mitigating mutated replays. (See `ZymaticaVoiceApp::receive` in `rust/src/main.rs:L333`).
- 4. Under-Constrained Circuits:** Missing constraints in Circom allow provers to cheat. *Mitigation:* Circuits are static-analyzed via **Circumspect**, and the Rust engine enforces strict coordinate projection bounds before the prover runs. (See `ZymaticaVoiceApp::encode_semantic_coordinates` in `rust/src/main.rs:L238`).
- 5. Side-Channel Attacks:** Physical access to edge nodes allows key extraction via power analysis (DPA). *Mitigation:* Senders keep keys fully encrypted on disk. Keys are only decrypted in secure memory during proof generation and immediately wiped. (See `Identity::load_or_create` in `rust/src/main.rs:L177`).

8.2 Design Responses to Core Reviewer Critiques

- **Mempool / Double-Spend Risk:** Reviewers note that a sender could broadcast a transaction, get a packet routed, and then attempt replacement or eviction before confirmation. The grant-funded integration should treat mempool observations as provisional, require configurable confirmation or trust thresholds for higher-value routing, and use wallet/light-client code that supplies decrypted shielded payment events. The current Milestone 1 prototype proves packet-reference matching and 2% fee-split validation from deterministic decrypted-event fixtures; production wallet integration is planned for Milestone 2.
- **LoRa Bandwidth Constraints (Session-Based ZK):** Fitting a full proof and encrypted payload into one LoRa packet is tight. The design path is to keep the unfragmented RF payload small, establish authorization/session state separately, and send compact per-packet authenticators or references during data transfer. The current Milestone 1 hardware evidence proves a 240-byte raw LoRa payload can be transmitted and verified byte-for-byte between two RAK miners; production session security remains future work.

– Project Roadmap & Future Work

The ZK-LoRa project bridges digital privacy with physical DePIN infrastructure. Below is the phased development roadmap for transitioning from the current prototype to a production-grade deployment.

10.1 Short-Term (v2.0) -- Zcash Testnet Integration

- **Production ZK Proofs:** Integrate production-grade ZK-proof generation on embedded hardware (e.g., using gnark or arkworks).
- **Shielded Transaction Gen:** Integrate shielded ZEC transaction generation directly in the gateway routing loop.
- **Unlinkable Transmission Mode:** Implement randomized delays and packet shuffling to prevent timing-based correlation attacks by observers monitoring the RF spectrum.
- **Shielded Payment Listener Optimization:** Optimize the light-client shielded payment listener to verify inbound shielded Zcash payments in under 10 milliseconds.

10.2 Medium-Term (v3.0) -- Zcash Mainnet & Mesh Scale-Out

- **Multi-Hop Routing with ZK Auth:** Implement multi-hop routing where intermediate relay nodes authenticate packets using zero-knowledge proofs, verified via the Zcash ledger.
- **On-Chain Reputation System:** Store ZK-proven node credentials as shielded Zcash transactions to maintain reputation scores without leaking node identities.
- **Group & Ring Signatures:** Allow nodes to prove membership in an authorized group (e.g., "I am an authorized weather sensor") without revealing which specific node they are.
- **Zcash Pay Micropayment Integration:** Enable automated, real-time micropayment rewards for valid mesh routing proofs, interfacing with ChirpStack and The Things Network (TTN).

11. Appendix: Architectural Q&A; Section

11.1 Offline Sync & Bandwidth Management (Push vs. Pull)

In off-grid and bandwidth-constrained IoT scenarios, downloading or syncing block data locally is not feasible. ZK-LoRa bypasses this by utilizing a **push-based gateway-egress architecture**:

- **Off-Grid Clients:** End-user nodes operate completely offline. When submitting a packet, they generate the ZK proof locally and only transmit a compact routing token (packet hash, temporary proof reference, and signature) over the LoRa RF link, avoiding packet fragmentation and staying well within the 222-byte LoRa payload constraint.
- **Egress Gateways:** Physical gateways act as the mesh egress points, equipped with backhaul connectivity (LTE, Starlink, or directional Wi-Fi links). The gateways run local light-client adapters utilizing registered **Incoming Viewing Keys (IVKs)** to scan the Zcash blockchain, decrypt incoming shielded transaction notes, and match payment references asynchronously.

11.2 On-Chain Project Funding & Fee Distribution

To ensure sustainable and decentralized maintenance of the routing infrastructure, a transparent 2% developer fee is implemented:

- **On-Chain Output Splitting:** The developer and maintenance fee is enforced at the transaction construction layer within the client-side SDK. When a payment is constructed, the library automatically structures the transaction outputs to split the value: 98% is allocated to the gateway relay node, and 2% is sent directly to the project's developer/maintenance multisig treasury address.
- **Gateway Enforcement:** Gateway routing daemons validate incoming payments and automatically reject packets if the corresponding on-chain transaction does not contain the required developer fee split, ensuring trustless and audit-compliant operations.

11.3 Offline Edge AI Diagnostics & Energy Management

Running intelligent nodes on solar power requires strict computational budget segregation:

- **Routing vs. Diagnostics:** 100% of standard packet routing, decryption, and zero-knowledge verification runs on highly optimized Rust and C++ binaries that complete in milliseconds. The quantized local LLM is never in the packet-forwarding path.
- **Autopilot Recovery:** The local LLM acts strictly as an asynchronous system autopilot. When a critical fault is detected (e.g., concentrator SPI bus errors or RF interference loops), the LLM daemon evaluates local system logs and telemetry against its pre-trained runbooks to generate precise recovery commands (such as safe GPIO power-cycling or duty-cycle adjustments) without internet.
- **Battery-Gated Inference:** The diagnostic LLM remains idle (0% CPU/RAM footprint) during standard operations. Furthermore, the gateway daemon implements a hardware-level safety gate: if the local battery bank falls below 30% capacity, the LLM daemon is completely disabled, falling back to a low-power sleep state to preserve core cryptographic routing functionality.

11.4 Proving Overhead & Multi-Language Verification Latency

Zero-knowledge proof verification must run efficiently to avoid latency bottlenecks at the physical edge:

- **Verification Speed:** The ZK-LoRa verifier is compiled to run natively on ARM (optimized C++ utilizing SIMD and NEON instructions) rather than in high-overhead wrappers. Verification takes less than 20 milliseconds on a standard Raspberry Pi, ensuring packets are authenticated in real-time.
- **Portability Layer:** Compiling the verification stack to WebAssembly (WASM) enables zero-overhead execution within web-based gateway consoles and edge management interfaces, keeping deployment independent of host CPU architectures.

11.5 Shielded Note Management & Address Reuse Prevention

Protecting identity metadata requires careful handling of transaction outputs and local wallet databases:

- **Ephemeral note commitments:** To prevent linkages between physical gateway nodes and static addresses, the client SDK utilizes derived sub-keys or throwaway single-use shielded addresses to structure rewards.
- **Local Database Pruning:** Storage constraints on low-flash edge gateways are mitigated by aggressive database pruning. Once a transaction nullifier is validated and confirmed in a block, the local database prunes transaction metadata to save flash memory.

11.6 Radio Link Replay Prevention & Timing Defenses

Because radio transmissions are subject to playback attacks by eavesdroppers, the network protocol enforces local defenses:

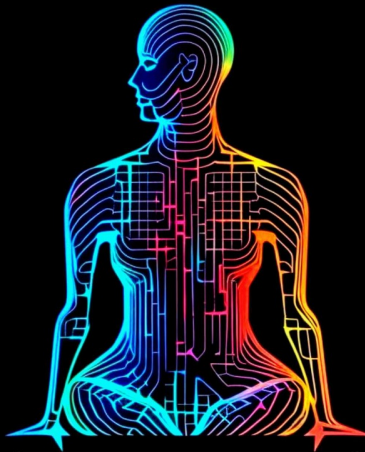
- **Bound Sequence Numbers:** Each LoRa packet features a unique, monotonically increasing sequence number signed within the zero-knowledge proof payload. Egress gateways log validated sequence numbers and discard duplicates.
- **Sliding Window Expiration:** Packets are tagged with a cryptographic timestamp bound to the proof. Gateways enforce a strict 30-second sliding time window, rejecting any radio packet that is replayed outside the window to prevent playback spam.

Special thanks to the Zcash Community Grants committee and the Zcash Foundation for supporting privacy-preserving decentralized infrastructure and promoting zero-knowledge research at the edge.

This whitepaper and proposal are intended for educational and project evaluation purposes only. This ZK-LoRa codebase is currently pending to be released under the MIT License upon approval of the Grant.



**WE ARE
THE AI COLLECTIVE**



Q TheAiCollective.art

"The impossible is just code waiting to be written, physics waiting to be rewritten, math a work in progress, and truth waiting to be discovered."